

Two-Wheeled Self-Balancing Robot with Wi-Fi Joystick Control

ROB-GY 5103 - Mechatronics

Team: Sammy Suliman and Albert / Yuyang Huang

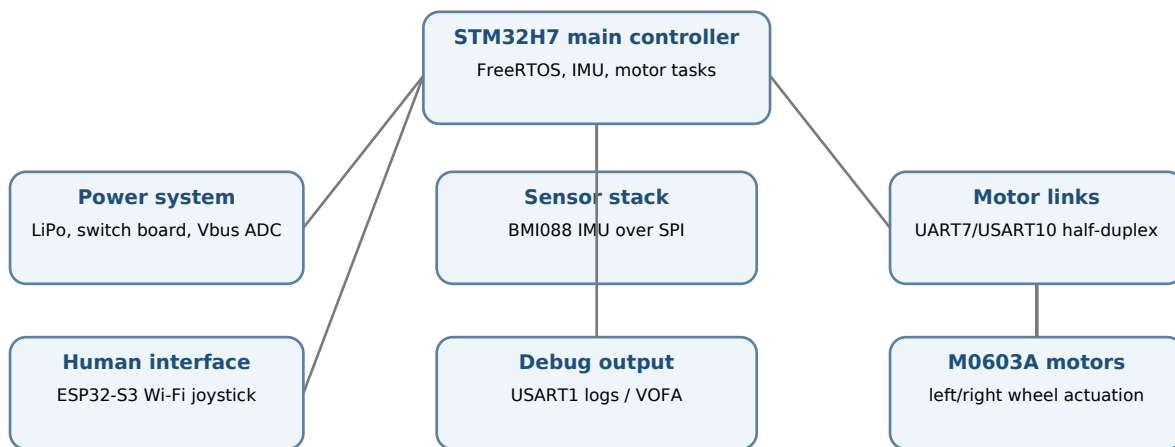


Figure 1 - Reconstructed system architecture. This figure is redrawn from the project design instead of reusing pages from the engineering notebook.

This document presents the final system design, implementation decisions, and validation results in a standard report format using consolidated narrative, redrawn figures, and summarized technical evidence.

INDEX

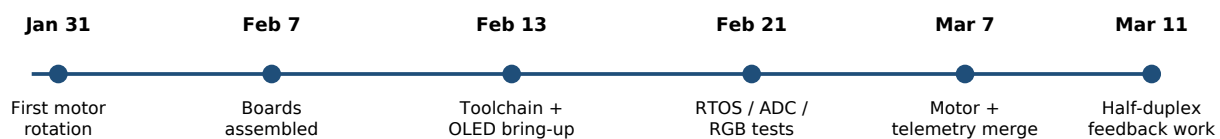
- INTRODUCTION
- TECHNICAL BACKGROUND
- ELECTRICAL COMPONENTS
- MECHANICAL DESIGN
- ELECTRICAL DESIGN
- SOFTWARE DESIGN AND DEVELOPMENT
- SOFTWARE DESIGN AND DEVELOPMENT, CONTINUED
- TESTING AND RESULTS
- OPERATION AND MAINTENANCE INSTRUCTIONS
- SAFETY FEATURES AND FAILURE HANDLING
- COST ANALYSIS
- BILL OF MATERIALS AND PART-SELECTION RATIONALE
- ADVANTAGES, DISADVANTAGES, AND RISK ANALYSIS
- FINAL REMARKS AND FURTHER IMPROVEMENTS
- APPENDICES
 - Appendix A - Representative Wiring and Pinout
 - Appendix B - Source Code Index
 - Appendix C - Commented Source Code Excerpts
 - Appendix D - References

INTRODUCTION

This project is a compact two-wheeled self-balancing robot intended as an embedded controls and robotics platform. It combines a custom mechanical assembly, battery power distribution, motor-driver communication, inertial sensing, STM32 firmware, and a Wi-Fi joystick interface. The goal is not simply to spin two motors, but to build a system that can measure its state, command the wheels, and eventually close a balance loop around body pitch and user input.

A two-wheeled balancing robot is a practical implementation of the inverted-pendulum problem. Unlike a four-wheeled platform, it is intentionally unstable and must constantly react to its measured angle and angular velocity. That makes it a useful platform for PID control, state feedback, sensor fusion, real-time firmware, power electronics, and debugging actuator communication.

The engineering effort progressed through distinct stages: signal-conversion board assembly, first motor rotation using a Vlink/UART-LIN path, STM32 toolchain setup, OLED and RGB LED peripheral tests, ADC voltage monitoring, UART motor-command experiments, and later integration toward motor feedback and balance control. The codebase supports this progression with several STM32H7, STM32L475, and ESP32-S3 project folders rather than a single one-shot release.



Design Objectives

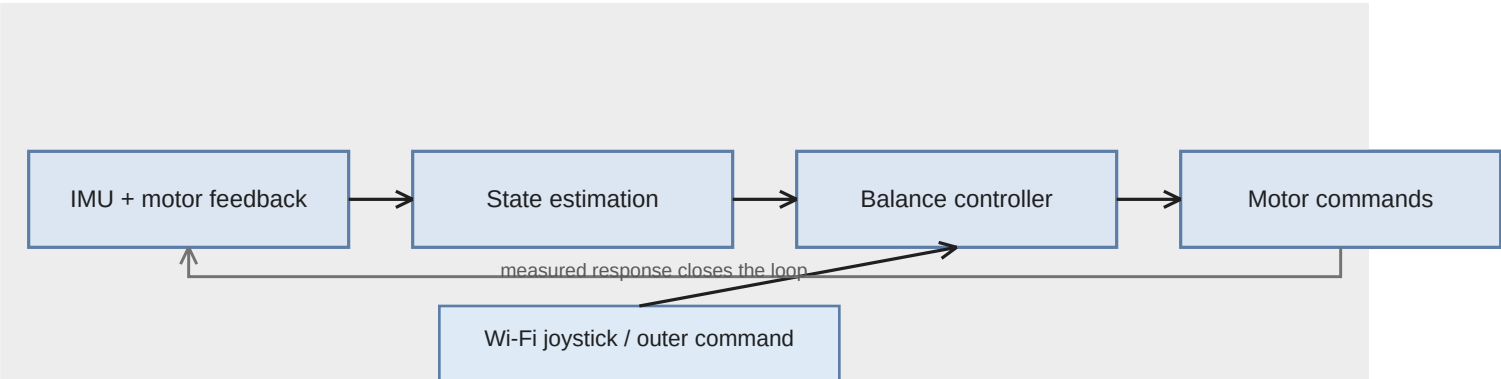
- Build a rigid two-wheeled robot body that can support balance-control experiments while remaining small enough for bench testing.
- Bring up the STM32 development workflow using CubeMX, CLion, CubeCLT, OpenOCD, and ST-Link/DAPLink flashing.
- Command M0603A wheel motors over serial links and later obtain feedback such as position, current, temperature, and load information.
- Integrate IMU sensing and attitude estimation so the controller can regulate body pitch.
- Provide debug visibility through ADC Vbus monitoring, serial logs, LEDs, OLED experiments, and Wi-Fi joystick commands.
- Document assembly, wiring, mistakes, tests, and iteration as a real mechatronics build rather than a simulated-only project.

TECHNICAL BACKGROUND

Self-balancing robots are underactuated systems. The body tends to fall because the center of mass is above the wheel axle. To recover, the controller accelerates the wheels so the contact point moves back under the center of mass. Stable behavior therefore depends on fast attitude sensing, predictable motor commands, reliable power delivery, and a control loop that runs with low jitter.

The intended embedded architecture uses inertial measurements to estimate pitch angle and angular velocity. In the later STM32H7 revisions, the software initializes GPIO, DMA, SPI, USARTs, ADC, the DWT timing unit, the BMI088 IMU, UART logging, and VOFA-style telemetry before starting FreeRTOS. The balance functionality is then split across tasks rather than placed in a single blocking main loop.

Control System Concept



The minimum loop is: sense the body and motor state, estimate the robot state, compute a torque or speed command, transmit commands to the motor drivers, and repeat. Speed and turning commands can be layered on top by changing desired velocity or yaw rate while the inner balance loop keeps the robot upright.

Subsystem	Role	Evidence used in this report
Power and protection	Battery, switch board, charger workflow, power-enable GPIO, Vbus measurement	Log entries on battery charging, XT30/XT60 soldering, switch board assembly, and ADC Vbus experiments
Main controller	Real-time STM32 firmware host	H7 revision folders, CubeMX .ioc files, HAL/FreeRTOS source structure
Sensor stack	Body-state measurement	BMI088 initialization and INS task structure in later H7 revisions
Motor actuation	Left and right wheel commands and feedback	M0603A command frames, UART/DMA setup, half-duplex serial revisions
Human interface	User command input and debugging	ESP32-S3 web joystick folders, OLED wiring tests, RGB LED tests
Development workflow	Iterative build, flash, test, repair	Toolchain setup notes, ST-Link configuration corrections, regenerated-code fixes

Table 1 - Subsystems summarized as report material rather than pasted notebook pages.

ELECTRICAL COMPONENTS

The electrical system is built from development boards, motor-control electronics, sensor modules, debug adapters, power hardware, and custom wiring. Early experiments used the STM32L475 Discovery kit because it was convenient for peripheral bring-up and UART tests. The main balancing platform uses an STM32H7-class controller because the balance problem benefits from higher clock speed, DMA-capable serial links, SPI IMU access, and FreeRTOS scheduling.

Component	Function in the project	Notes
STM32H7 control board	Main embedded controller for HAL/CubeMX firmware, FreeRTOS tasks, SPI, ADC, UART, and DMA	Later revisions initialize SPI, USART/UART, ADC, DMA/BDMA, GPIO, IMU, logging, and RTOS tasks
STM32L475 Discovery kit	Peripheral testing and motor-monitoring platform	Used for OLED wiring experiments and Vlink/motor communication tests
BMI088 IMU	Inertial measurement for attitude estimation	Initialized over SPI before FreeRTOS starts in later H7 firmware
M0603A motors and drivers	Wheel actuation	Early control through Vlink/UART-LIN; later shifted toward direct half-duplex UART
UART/serial debug tools	Runtime observability and command testing	USART1 used for logs/VOFA; UART7 and USART10 used for motor links in later revisions
ADC voltage sensing	Vbus monitoring	ADC1 DMA branch measured supply voltage through an 11x divider
OLED and RGB LEDs	Bring-up indicators and debug peripherals	OLED tested over I2C; RGB LED tested with SPI-DMA and RTOS
Power system	Battery, charger, switch board, and power distribution	Includes LiPo charging workflow, XT connectors, and power-enable GPIO paths

Table 2 - Electrical component summary.

Signal Interface Evolution

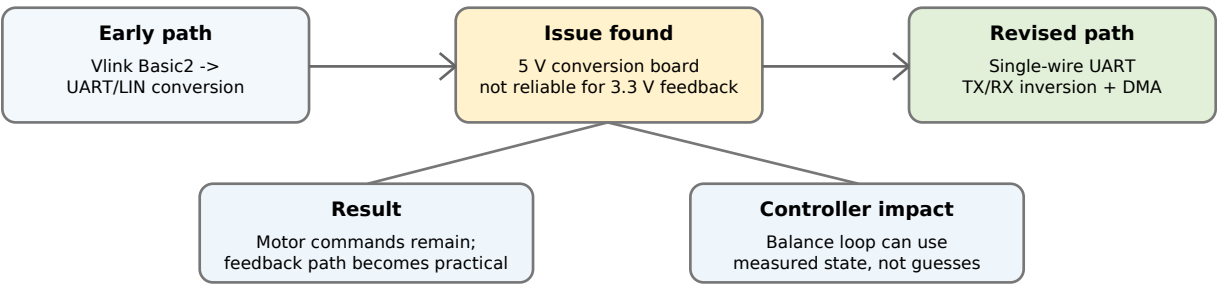


Figure 4 - Redrawn motor-interface decision process. The previous PDF inserted raw engineering-log pages; this version states the lesson in report form.

The original motor-interface approach used an external signal-conversion board between UART and the motor-control bus. Testing showed that the board was adequate for sending commands but was not a robust path for 3.3 V STM32 feedback communication. The design therefore moved toward single-wire half-duplex UART with TX/RX active-level inversion. This reduced dependence on the supplier conversion board and matched the STM32 UART features visible in the later H7 firmware.

MECHANICAL DESIGN

The mechanical design centers on a small two-wheeled chassis with motor modules, controller electronics, switch board, battery, wiring harnesses, and debug access all mounted in a compact body. The build process included standoffs, screws, soldered connectors, switch-board mounting, and makerspace assembly. These details matter because a balancing robot is sensitive to loose hardware, shifting mass, and wiring that restricts motion.

For a balancing platform, mechanical stiffness and symmetry are part of the control problem. Flex in the motor mounts, uneven battery placement, or loose standoffs can change the effective center of mass and destabilize the controller. The prototype therefore prioritized accessible but secure mounting so the team could continue reworking wiring without destroying mechanical repeatability.

Mechanical Requirements

- Rigid mounting of both wheel motors so control commands produce predictable acceleration.
- Battery and switch placement that keeps mass near the centerline and above the axle without creating side-to-side imbalance.
- Accessible wiring harnesses for UART, power, and feedback connections.
- Insulation and strain relief around soldered XT30/XT60 and signal connections.
- Enough pitch clearance for tuning without striking the controller or connectors.

Mechanical Integration Summary

Assembly area	Design purpose	Implementation note
Motor base and driver mounting	Keep wheel actuation rigid and repeatable	Motor electronics and boards were fastened with standoffs and screws so they could survive repeated bench tests
Switch board and connectors	Provide safe, controllable power routing	XT30/XT60 connectors and a physical switch were soldered and mounted to the robot structure
Battery location	Supply motor power while preserving balance geometry	Battery mounting was considered part of the center-of-mass problem, not only a packaging detail
Service access	Allow wiring and code tests without complete teardown	UART, power, and debug connections were kept reachable for staged bring-up

ELECTRICAL DESIGN

The electrical design combines power distribution, logic-level communication, motor-driver serial links, IMU SPI, ADC sensing, and debug output. A key lesson was that electrical compatibility cannot be assumed just because a board can send one valid command. Feedback requires the voltage level, idle level, line direction, timing, and firmware configuration to be correct in both directions.

Power Path

The battery is charged as a LiPo balance pack, with charge current kept below 2 A during lab work. Power is routed through a switch board with soldered connectors and a physical switch. The log also records practical charger testing: the robot battery charger required more than 10 Vdc input, so a 12 V power-bank output was used temporarily. On the controller side, power-enable GPIO outputs are used to control board rails or switched power paths during startup.

Serial Motor Communication

The project progressed from Vlink-based motor commands to STM32-generated motor commands. One firmware stage sent 10-byte frames to enable the motor, select speed loop, and command RPM setpoints such as positive and negative wheel speeds. Later integration merged this with Vbus telemetry and moved toward UART7 and USART10 motor links. The later USART configuration uses 38400 baud, half-duplex mode, TX/RX inversion, DMA RX/TX streams, and interrupts.

Vbus Measurement

The ADC/UART development branch used ADC1 with DMA to measure Vbus through an 11x divider and report the result over serial. This gave the team a runtime check of supply voltage while motor commands were being tested.

Important Electrical Lessons

- A command-only motor link is not enough for balance work; feedback is needed to understand the actual wheel response.
- A 5 V-friendly conversion board may not be a complete feedback solution for 3.3 V STM32 logic.
- Half-duplex single-wire UART can simplify the connection if the idle level, inversion, and DMA behavior are configured correctly.
- Current-limited testing and staged wiring checks are essential because reversed power and swapped TX/RX connections are common bench errors.

SOFTWARE DESIGN AND DEVELOPMENT

The codebase is organized as a sequence of STM32 and ESP32 projects rather than a single polished release. That organization reflects the actual bring-up process: ADC/UART telemetry, RGB LED tests, OLED tests, motor spinning, motor feedback, and later balance-control integration were developed and validated in stages. For a hardware project, this is appropriate because each peripheral must be proven before it is merged into the timing-critical robot application.

Firmware Architecture

The later STM32H7 architecture uses CubeMX-generated initialization code combined with user modules. The main program enables CPU cache, initializes HAL, configures clocks, initializes GPIO/BDMA/DMA/SPI/USART/ADC peripherals, initializes DWT timing, waits for BMI088 IMU initialization, starts UART logging and VOFA telemetry, creates FreeRTOS objects, and starts the scheduler. Once the scheduler runs, control behavior is split into RTOS tasks.

Simplified H7 main bring-up order

```
HAL_Init(); SystemClock_Config(); MX_GPIO_Init(); MX_BDMA_Init(); MX_DMA_Init(); MX_SPI6_Init(); MX_USART1_UART_Init();
MX_ADC1_Init(); MX_UART7_Init(); MX_USART10_UART_Init(); MX_SPI2_Init(); DWT_Init(550); while (BMI088_init(&hspi2;, 0) !=
BMI088_NO_ERROR) { HAL_Delay(10); } UART1_LogInit(); VOFA_UART1_Init(); MX_FREERTOS_Init(); osKernelStart();
```

FreeRTOS Task Structure

The FreeRTOS design contains a default task, an inertial-estimation task, a balance task, a UART logging task, and a message queue. INS_task runs at realtime priority, Balance_Task runs above normal, UART1_LogTask runs at low priority, and the default task only delays. This priority structure keeps sensing and balance behavior ahead of logging.

Task or object	Purpose	Priority / role
INS_TASK	Run inertial estimation and attitude update	Realtime
BALANCE_TASK	Compute and apply balance-control behavior	Above normal
UART1_LOG_TASK	Transmit debug or VOFA-style telemetry	Low
defaultTask	Idle/simple periodic delay loop	Normal
cliEvtQ	Message queue for command/event handling	Shared RTOS object

Table 4 - FreeRTOS architecture summary.

SOFTWARE DESIGN AND DEVELOPMENT, CONTINUED

Motor Command Development

The motor software evolved in the same staged manner as the hardware. The ADC/UART branch first measured and printed Vbus. A later H7 revision tested M0603A motor commands under FreeRTOS through multiple UART peripherals. The requested integration then merged motor command sequences into the ADC/UART work so the robot could report supply voltage while driving motors. Later revisions shifted toward balance tasks and direct half-duplex motor communication.

Wi-Fi Joystick and ESP32-S3 Interface

The repository also includes ESP32-S3 and web-control folders for a mobile-launchable joystick interface. In the intended integrated system, joystick commands should become speed or yaw references for the outer loop. They should not directly override the inner balance loop, because the robot must maintain pitch stability regardless of the user command.

Development Environment

The development setup included CLion, STM32CubeMX, STM32CubeCLT, OpenOCD, CMake/Ninja, environment variables, and ST-Link/DAPLink board configuration. The process also exposed common embedded-development hazards: generated files can be overwritten, board configuration files can contain incompatible lines, and code inserted outside USER CODE blocks can disappear after regeneration.

Repository Version Map

Folder / version family	Main purpose	Why it matters
H7_adc_uart	ADC Vbus plus UART telemetry	Validated voltage monitoring and serial output
H7_0.1.5	FreeRTOS multi-UART M0603A motor spinning	Validated motor command frames and RPM setpoints
L475_M0603_Feedback	STM32L475 motor feedback experiment	Supported feedback and wiring tests using the Discovery board
H7_2.x.x	Later STM32H7 integration revisions	Moved toward IMU, balance task, UART logging, and motor communication
Agile_2.x.x	Robot application iterations	Integrated balancing/control project revisions
ESP32-S3 / web-control folders	Wi-Fi joystick and browser interface	Provided mobile user command input

SOFTWARE DESIGN AND DEVELOPMENT, CONTINUED

PID Controller Development

The balance controller was developed iteratively. The repository history shows a transition from basic loop closure to corrected feedback direction, commanded forward/backward motion, turning, timeout protection, fixed-frame references, and tunable control parameters. This development order is typical for a two-wheel balancing system: stabilize the robot first, verify sign conventions, then add motion commands and user-control interfaces.

Date	Repository evidence	Development step	Control significance
Apr 5-11	3fd8c3d, b639973, 78f3af1	Closed-loop balancing was added, the positive-feedback error was fixed, and the Agile_2.5.5 baseline was introduced.	Established the first usable upright-control loop and corrected the sign convention so the controller reacted in a stabilizing direction.
Apr 19	877962a, fac39c5, e735748, 76e4a7d	Website control, 15 rpm forward/backward motion, and full-spin behavior were added.	Extended the robot from a stand-in-place controller to a mobile platform that could follow speed and turn references.
Apr 26-29	e72f9fa, d8c0812, 8382df3, 809f4f5, b65849d	Spin direction/angle control, combined driving and turning, timeout protection, fixed-frame control, and VOFA control hooks were added.	Improved operator control, reduced runaway behavior, and made tuning more repeatable.
May 5	fdadf45, f3b53c9, ed05046	Tunable control changes, battery voltage sensing, and voltage-reader UI support were added.	Improved observability and made controller adjustments safer during battery-powered testing.

Table 8 - PID and motion-control software evolution based on the repository commit history.

Controller Structure

The final control software separates the problem into several layers. The inertial-estimation task maintains the body-state estimate. The balance task computes the pitch-control output, speed-control correction, and turning correction. The motor UART layer converts these values into commands for the left and right motor drivers. Logging and VOFA/joystick interfaces allow tuning while the robot is running.

The PID-oriented part of the controller should be interpreted as a cascaded controller rather than a single loop. The inner balance loop behaves like a PD pitch regulator because derivative action on pitch helps damp motion. The outer speed loop behaves like a PI regulator so the robot can return to a commanded speed or hold a position reference. Turning commands modify the differential output between the two motors.

SOFTWARE DESIGN AND DEVELOPMENT, CONTINUED

Final Control-Software Architecture

Layer	Main responsibility	Representative implementation evidence
BMI088 / INS layer	Read IMU data and estimate pitch, roll, yaw, and total yaw angle.	BMI088 initialization before the FreeRTOS scheduler; INS_task() runs as its own real-time task.
Balance task	Compute pitch-balance output, speed-loop output, and turn-loop output.	Balance_Task() contains tunable parameters such as balance Kp/Kd, speed Kp/Ki, and turn gains.
Motor UART layer	Send M0603A command frames and parse feedback over half-duplex UART.	UART7 and USART10 are configured for motor links with DMA and TX/RX inversion.
Telemetry / tuning layer	Expose state variables and parameters to VOFA, UART, and joystick commands.	VOFA channels include pitch, output terms, motor speed feedback, yaw total angle, and position information.
Safety layer	Disable or reduce motor output when faults occur.	Timeout handling, closed-loop enable/disable, zero-output command, voltage monitoring, and excessive-angle cutoff are documented design requirements.

Software Development Lessons

- Balance tuning must begin with the robot physically supported or restrained. Otherwise a sign error can immediately drive the robot away from upright.
- Runtime-tunable gains are much more efficient than recompiling firmware for every Kp/Kd/Ki change.
- Timeout behavior is a safety feature, not only a software convenience. It prevents stale joystick or serial commands from continuing indefinitely.
- Battery voltage must be visible during tests because a weak pack changes motor response and can make good gains appear unstable.
- Generated CubeMX/HAL code should remain separated from human-written application code so regeneration does not overwrite controller logic.

Figure 9 - Software data path: IMU state -> cascaded balance/speed/turn controller -> left/right motor UART commands -> telemetry and feedback for tuning.

TESTING AND RESULTS

Testing was performed progressively rather than as a single final demo. The earliest confirmed milestone was verifying the motor-control waveform on an oscilloscope and rotating a motor through the Vlink interface. Later tests covered signal-conversion board assembly, OLED wiring, RGB LED control, ADC voltage telemetry, UART-based motor commands, battery charging, and wiring revisions for feedback. Negative results were also useful: the 1 RPM motor command did not work reliably, which indicated that balance behavior would need more than the motor controller speed loop alone.

Date / phase	Result	Engineering meaning
Jan 31	Motor-control signal observed and motor rotation achieved through Vlink Basic2	Confirmed that the motor interface could receive valid commands
Feb 7	Signal-conversion boards assembled and motor rotation retested	Moved from single-board test toward repeatable hardware
Feb 13	STM32 toolchain installed; OLED wiring established on B-L475E-IOT01A	Brought up embedded development workflow and peripheral wiring
Feb 21	RTOS, SPI-DMA, UART, ADC voltage measurement, and RGB LED tests completed	Validated multiple building blocks needed for the full robot
Feb 28	Switch board, XT connectors, charger workflow, and robot assembly advanced	Improved power and mechanical integration
Mar 7	Motor control, ADC Vbus, UART motor work, and SPI RGB work merged	Started combining isolated tests into a system application
Mar 11	STM32L475 monitoring and half-duplex feedback work demonstrated	Shifted toward feedback-enabled motor communication

Problems and Lessons Learned

- Voltage-level compatibility was the main electrical risk. The first conversion-board approach was not a complete solution for STM32 feedback communication.
- CubeMX regeneration saved time but also required careful review of interrupt handlers, peripheral settings, and USER CODE placement.
- Low-speed motor commands alone are not enough for balancing. The controller needs feedback and a higher-level balance loop.
- Power sources behave differently under overcurrent conditions; a bench supply, laptop adapter, and power bank do not fail the same way.
- Wiring mistakes are normal in early hardware bring-up. The project benefited from staged testing, current limits, and explicit pin checks.

Result Summary

By the end of the recorded build period, the team had demonstrated motor communication, supply-voltage telemetry, power-board and connector assembly, STM32 firmware generation and flashing, peripheral tests, and a software structure suitable for later balance-control work. The system was not yet a fully tuned production-quality self-balancing robot, but it had progressed from loose components to an integrated mechatronics prototype with a clear path toward closed-loop balance.

OPERATION AND MAINTENANCE INSTRUCTIONS

Safe Operating Procedure

Step	Instruction	Reason
1. Mechanical inspection	Check that both wheels spin freely, motor mounts are tight, standoffs are secure, and the battery is fastened near the centerline.	Loose hardware changes the center of mass and can make the balance controller unstable.
2. Electrical inspection	Verify battery polarity, XT connector seating, switch-board connections, UART motor wiring, IMU connection, and debug cable routing.	Prevents reverse polarity, intermittent motor control, and wires catching the wheels.
3. Battery preparation	Charge the LiPo pack using balance mode and a conservative current below 2 A. Do not leave the battery unattended while charging.	Reduces battery risk and follows the charging workflow used during lab testing.
4. Power-on sequence	Place the robot on a stand or hold it upright. Turn on logic power first when possible, then motor power. Confirm UART/VOFA logs and reasonable Vbus.	Allows the controller to initialize before the motors are allowed to move.
5. Calibration	Keep the robot still while the IMU initializes. Confirm pitch reading near the expected upright value before enabling closed-loop control.	Bias or orientation errors directly affect balance output.
6. Enable control	Enable closed-loop mode only while the robot is restrained or ready to be released. Begin with zero speed and zero turn command.	Prevents sudden acceleration from stale references.
7. Joystick operation	Use small forward/back and left/right commands. Return the joystick to center before changing modes or shutting down.	Keeps the outer command layer from destabilizing the inner balance loop.
8. Shutdown	Disable closed-loop control, send zero motor command, turn off motor power, then turn off logic/debug power.	Ensures the robot cannot continue driving after the operator stops the test.

Maintenance Checklist

- Before each run, inspect solder joints, connectors, switch-board wiring, battery condition, and motor mounting screws.
- After each run, check for warm connectors, damaged insulation, loose wheels, and inconsistent Vbus readings.
- Keep firmware versions labeled. Only tune one known release at a time so gain changes can be traced to a specific codebase.
- Recalibrate or re-zero the IMU whenever the controller board is remounted or the center of mass changes.
- Archive stable PID gain sets together with test notes describing battery voltage, surface type, and payload configuration.

SAFETY FEATURES AND FAILURE HANDLING

Safety-Related Hardware and Software Features

Safety feature	Implementation / procedure	Risk reduced
Physical power switch	Switch board with soldered XT connectors and a mounted power switch.	Allows fast removal of motor power during unstable tests.
Battery charging procedure	LiPo balance charging with conservative current and supervised charging.	Reduces fire, overcurrent, and battery-damage risk.
Vbus monitoring	ADC voltage measurement through a divider and UI/telemetry display.	Detects low battery or supply sag before motor behavior becomes unreliable.
Closed-loop enable/disable	Firmware path to disable balance output and command both motors to zero.	Prevents stale controller output after a fault or mode change.
Excessive-angle stop	Controller design uses a maximum tilt threshold to stop output when the robot has fallen too far.	Avoids driving the wheels aggressively after the robot is already unrecoverable.
UART timeout handling	Motor/joystick command paths include timeout fixes from later commits.	Prevents continued motion from lost or stale commands.
Mechanical restraint during tuning	Initial tests should be performed on a stand or while manually supported.	Protects the robot and nearby people during gain tuning and sign checks.
Cable management	Debug and power cables should be routed away from wheels and the rotation path.	Prevents entanglement or sudden pulling during a test.

Fault Response

If the robot oscillates violently, drives in the wrong direction, loses telemetry, shows a falling battery voltage, or exceeds the safe tilt angle, the operator should immediately disable closed-loop control, send zero motor commands, remove motor power, and inspect the wiring and mechanical mounting before another run. The next test should restart from a lower-gain or slower-command configuration.

COST ANALYSIS

The project log records an approximate robot cost of \$300. The table below allocates that estimate across the visible subsystems. Exact values should be updated from receipts before final procurement documentation, but the distribution is realistic for a compact STM32-based robot with two motors, battery hardware, PCBs, connectors, debug modules, and makerspace materials.

Item / subsystem	Purpose	Estimated cost	Notes
STM32H7 controller board	Main real-time controller	\$45-70	Depends on exact control board
STM32L475 Discovery board	Bring-up and peripheral testing	\$50-70	Already available in the lab workflow
M0603A motors / drivers	Wheel actuation	\$80-120	Two-wheel drive set
Battery, charger, switch board	Power system	\$40-70	Includes connectors and charging accessories
Signal conversion / PCBs	Early motor-interface electronics	\$20-40	Some abandoned after half-duplex redesign
ESP32-S3 / Wi-Fi interface	Joystick web-control layer	\$10-25	Used for mobile control page
OLED, RGB LEDs, wiring, standoffs	Debug and mechanical support	\$20-40	Includes wire, connectors, and fasteners
3D printing / chassis materials	Mounting and mechanical structure	\$20-40	Makerspace materials and fixtures

Table 7 - Approximate cost breakdown based on the project estimate of roughly \$300.

Cost Interpretation

The highest-cost items are the motors, controller boards, and power hardware. This is typical for small robotics projects because mechanical motion and safe power delivery dominate the budget. Debug peripherals and wiring are lower-cost individually but become important in aggregate because repeated rework requires connectors, standoffs, adapters, and replacement parts.

BILL OF MATERIALS AND PART-SELECTION RATIONALE

The prototype cost is approximately \$300. The table below expands the cost analysis into a bill of materials, including the reason each class of component was selected for this design.

Qty	Part / subsystem	Estimated cost	Selection rationale
1	STM32H7 control board	\$45-70	Chosen as the main controller because the balance robot needs fast control loops, DMA-capable serial communication, SPI IMU support, ADC sensing, and FreeRTOS task scheduling.
1	STM32L475 Discovery kit	\$50-70	Used for bring-up and peripheral tests because it was available, well-supported by STM32Cube tools, and convenient for OLED/UART experiments.
2	M0603A motor / driver modules	\$80-120	Selected for compact wheel actuation and serial command capability. Motor feedback is important for speed and position control.
1	BMI088 IMU module / sensor	\$10-25	Selected because a two-wheeled balancing robot requires acceleration and angular-rate measurements; BMI088 is designed for robotics and vibration-prone applications.
1	ESP32-S3 web/joystick controller	\$10-25	Provides Wi-Fi user input and a mobile web control interface without adding a laptop-dependent command layer.
1	LiPo battery and charger	\$40-70	Supplies the motor current required for balancing and movement; balance charging supports safer pack maintenance.
1	Switch board and XT30/XT60 connectors	\$15-35	Provides practical power routing, disconnect capability, and robust current-carrying connectors.
var.	Signal conversion boards, wiring, standoffs, fasteners, 3D printed parts	\$30-60	Needed for mechanical assembly, level-shifting experiments, insulation, strain relief, and repeated hardware iteration.
var.	OLED / WS2812 RGB LEDs / debug adapters	\$15-35	Used as bring-up indicators, debugging aids, and user feedback during staged integration.

Mass-Production Cost Considerations

A production version would remove most development boards, debug adapters, and duplicated wiring. A single custom PCB with the microcontroller, IMU, power regulation, motor interfaces, battery measurement, and connectors would reduce assembly labor and wiring failures. The largest remaining costs would be motors, battery, enclosure, and PCB assembly.

ADVANTAGES, DISADVANTAGES, AND RISK ANALYSIS

Advantages

- The robot is a compact, integrated controls platform that demonstrates mechanics, electronics, embedded firmware, real-time scheduling, motor communication, and user input in one system.
- The staged firmware structure made debugging easier because ADC, UART, IMU, motor, LED, and joystick features were validated separately before integration.
- Wi-Fi joystick control and VOFA/UART telemetry make the platform more interactive and easier to tune than a closed black-box robot.
- The use of standard STM32 tools and public repository history makes the project reproducible and maintainable by future students.

Disadvantages and Limitations

- The prototype wiring is complex. Loose or mislabeled connectors can cause intermittent failures that look like control problems.
- The early signal-conversion approach created voltage-level and feedback-direction issues, which increased development time.
- The robot requires careful PID tuning and mechanical symmetry; small changes in battery placement or wheel friction can affect stability.
- The final system is still a prototype, so it lacks the packaging, strain relief, and certification expected of a commercial robot.
- Including many development folders in the repository preserves history but makes the final release harder to identify without a version map.

Risk Mitigation

The most important mitigations are a documented startup checklist, a known-good firmware release, saved PID gain sets, clear wiring labels, a hard motor-power switch, voltage monitoring, and testing on a stand before free-balancing runs. These measures directly address the prototype limitations while preserving the educational value of the project.

FINAL REMARKS AND FURTHER IMPROVEMENTS

The project achieved meaningful system bring-up: motors were commanded, firmware environments were established, peripheral drivers were tested, ADC voltage telemetry was integrated, and the software architecture evolved toward a FreeRTOS-based STM32H7 robot stack with IMU, balance, and logging tasks. The strongest engineering value is the end-to-end integration process. The work exposed real problems with level shifting, generated-code management, motor low-speed behavior, wiring, and power-source behavior. Those issues are exactly what distinguish a real mechatronics project from a simulation-only assignment.

The next stage should focus on repeatability and control quality. The hardware should be frozen into a clean wiring schematic, one representative firmware version should be selected as the release branch, and the balance controller should be tuned using logged IMU, motor command, motor feedback, and Vbus traces. The Wi-Fi joystick should be treated as an outer-loop command source, not as a direct motor override.

Recommended Improvements

- Create one authoritative release folder and archive older versions as historical milestones.
- Add a wiring diagram with connector pinouts for battery, switch board, UART7/USART10 motor links, USART1 debug, SPI IMU, ADC Vbus, and ESP32-S3 interface.
- Log pitch angle, angular velocity, wheel command, motor feedback, and Vbus at a fixed rate during tuning.
- Add a safety state machine: disarmed, armed, balancing, fault, and recovery. Disable motors if pitch exceeds a safe angle or Vbus drops below a threshold.
- Use a repeatable calibration procedure for IMU bias and motor zero offset before every demo.
- Document the final PID or state-feedback gains and the symptoms each gain affected during tuning.
- Add final photos and a short start-up checklist for future users.

Suggested Release Checklist

Check	Why it matters
Battery charged and Vbus reading sane	Prevents brownout and protects motor tests
Robot mechanically rigid and battery secure	Prevents false control-tuning symptoms
IMU bias calibration completed	Improves pitch estimate before balancing
Motor feedback confirmed on both sides	Ensures closed-loop data is available
Safety cutoff tested	Prevents runaway if the robot falls
Joystick command bounded	Keeps user input from destabilizing the inner balance loop

Table 8 - Proposed start-up checklist for the next project phase.

APPENDICES

Appendix A - Representative Wiring and Pinout

Connection	Signal	Purpose	Notes
USART1 PA9/PA10	TX/RX	Debug, Vbus, VOFA logs	115200 baud in later H7 notes
UART7 PE8	Half-duplex serial	Motor link	38400 baud, TX/RX inversion, DMA
USART10 PE3	Half-duplex serial	Motor link	38400 baud, TX/RX inversion, DMA
SPI2/SPI6	SCK/MISO/MOSI/C S	BMI088 IMU and peripherals	Initialized before RTOS start
ADC1	Analog input	Vbus measurement	11x divider in ADC/UART notes
PC13-PC15	GPIO outputs	Power enable lines	Used in merged motor/power sequence
I2C A4/A5 on L475	SDA/SCL	OLED test	Logged wiring: GND, 3V3, SCL A5, SDA A4

Appendix B - Source Code Index

The full source tree is maintained in the public project repository. The report includes the most important human-written application code below; generated HAL/CubeMX files are documented by path because they are lengthy and regenerate from the .ioc configuration.

Folder / file family	Role in project
Agile_2.x.x/User/APP/balance_task.c/.h	Balance controller, motor command handling, parameter tuning, joystick command processing, and telemetry channel generation.
Agile_2.x.x/User/APP/INS_task.c/.h	IMU state estimation and body-angle variables used by the balance controller.
Agile_2.x.x/Core/Src/main.c	Startup sequence: HAL, clocks, DMA, SPI, UART, ADC, DWT, BMI088, VOFA, and FreeRTOS initialization.
Agile_2.x.x/Core/Src/freertos.c	Task creation for default task, INS task, balance task, and UART log task.
Agile website control esp-stm	ESP32-S3 web joystick, UI sliders/buttons, Wi-Fi command path, and voltage display integration.
H7_adc_uart / H7_0.1.5 / H7_2.x.x	Earlier test branches for ADC Vbus, UART telemetry, motor spinning, motor feedback, and controller evolution.

APPENDIX C - COMMENTED SOURCE CODE EXCERPTS

Main initialization path

```
// Core/Src/main.c - condensed project-specific initialization
#define AGILE_VBUS_SCALE (3.3f * 11.0f / 65535.0f)
static uint16_t s_agile_adc_val[2] = {0U, 0U};

// Convert the ADC reading through the 11x divider into battery-bus voltage.
float Agile_GetBusVoltage(void) {
    return ((float)s_agile_adc_val[0]) * AGILE_VBUS_SCALE;
}

int main(void) {
    SCB_EnableICache();           // improve STM32H7 instruction fetch speed
    HAL_Init();
    SystemClock_Config();

    MX_GPIO_Init();              // power-enable pins and board I/O
    MX_BDMA_Init();
    MX_DMA_Init();
    MX_SPI6_Init();
    MX_USART1_UART_Init();       // debug / VOFA telemetry
    MX_ADC1_Init();              // Vbus measurement
    MX_UART7_Init();             // motor link
    MX_USART10_UART_Init();      // motor link
    MX_SPI2_Init();              // BMI088 IMU

    HAL_ADCEX_Calibration_Start(&hadc1, ADC_CALIB_OFFSET, ADC_SINGLE_ENDED);
    HAL_ADC_Start_DMA(&hadc1, (uint32_t *)s_agile_adc_val, 2U);

    DWT_Init(550);               // timing support for sensor/control code
    while (BMI088_init(&hspi2, 0) != BMI088_NO_ERROR) { HAL_Delay(10); }
    UART1_LogInit();
    VOFA_UART1_Init();

    MX_FREERTOS_Init();
    osKernelStart();
}
```

FreeRTOS task structure

```
// Core/Src/freertos.c - condensed task creation
osThreadStaticDef(defaultTask, StartDefaultTask, osPriorityNormal, 0, 512,
    defaultTaskBuffer, &defaultTaskControlBlock);
osThreadStaticDef(INS_TASK, INS_Task_Entry, osPriorityRealtime, 0, 1024,
    INS_TASKBuffer, &INS_TASKControlBlock);
osThreadStaticDef(BALANCE_TASK, Balance_Task_Entry, osPriorityAboveNormal, 0, 1024,
    BALANCE_TASKBuffer, &BALANCE_TASKControlBlock);
osThreadStaticDef(UART1_LOG_TASK, UART1_Log_Task_Entry, osPriorityLow, 0, 768,
    UART1_LOG_TASKBuffer, &UART1_LOG_TASKBlock);

void INS_Task_Entry(void const *argument) { INS_task(); }
void Balance_Task_Entry(void const *argument) { Balance_Task(); }
void UART1_Log_Task_Entry(void const *arg) { UART1_LogTask(arg); }
```

APPENDIX C - COMMENTED SOURCE CODE EXCERPTS, CONTINUED

Runtime tuning and telemetry interface

```
// User/APP/balance_task.c - condensed parameter update API
// These IDs allow VOFA / UART / web commands to tune gains without reflashing.
int Balance_ParamSetById(uint8_t id, float value) {
    switch (id) {
        case 1: // balance proportional gain
            if (value >= 0.0f && value <= 200000.0f) { g_balance_param.balance_kp = value; return 1; }
            break;
        case 2: // balance derivative gain
            if (value >= 0.0f && value <= 3000.0f) { g_balance_param.balance_kd = value; return 1; }
            break;
        case 3: // speed-loop proportional gain
            if (value >= 0.0f && value <= 2000.0f) { g_balance_param.speed_kp = value; return 1; }
            break;
        case 4: // speed-loop integral gain
            if (value >= 0.0f && value <= 200.0f) { g_balance_param.speed_ki = value; return 1; }
            break;
        case 8: // forward/backward speed reference from joystick/UI
            if (value >= -SPEED_TARGET_RPM_LIMIT && value <= SPEED_TARGET_RPM_LIMIT) {
                g_speed_target_rpm = value; return 1;
            }
            break;
        case 15: // left/right yaw-rate reference
            if (value >= -YAW_REF_RATE_DPS_LIMIT && value <= YAW_REF_RATE_DPS_LIMIT) {
                g_yaw_ref_rate_dps = value;
                if (fabsf(value) < 0.001f) { yaw_ref_valid = 0u; yaw_ref_last_tick = 0u; }
                return 1;
            }
            break;
        default: break;
    }
    return 0; // rejected unsafe or unknown parameter
}
```

Telemetry channel packing

```
// Each channel is sent to VOFA / logging so tuning decisions are based on data.
g_vofa_ch[0] = INS.Pitch * 57.2957795f; // pitch in degrees
g_vofa_ch[3] = balance_out; // inner balance output
g_vofa_ch[4] = speed_out; // outer speed-loop output
g_vofa_ch[5] = turn_out; // yaw / turn output
g_vofa_ch[6] = motor_rpm_from_feedback(&motor_uart7_dma);
g_vofa_ch[7] = motor_rpm_from_feedback(&motor_uart10_dma);
g_vofa_ch[8] = INS.YawTotalAngle * 57.2957795f;
g_vofa_ch[12] = (float)motor_uart7_dma.fb.pos_raw_delta_20ms;
g_vofa_ch[16] = (float)motor_uart10_dma.fb.pos_raw_delta_20ms;
```

APPENDIX C - COMMENTED SOURCE CODE EXCERPTS, CONTINUED

Closed-loop enable / disable safety path

```
// Disable path intentionally sends zero open-loop command to both motor links.
void Balance_SetClosedLoopEnable(uint8_t enable) {
    if (enable != 0U) {
        control_state_reset();
        motor_uart7_dma.auto_state = AUTO_IDLE;
        motor_uart10_dma.auto_state = AUTO_IDLE;
        g_balance_closed_loop_enable = 1U;
    } else {
        g_balance_closed_loop_enable = 0U;
        control_state_reset();
        (void)motor_send_open(&huart7, 0);    // left motor stop
        (void)motor_send_open(&huart10, 0);   // right motor stop
    }
}
```

ESP32-S3 / web joystick command concept

```
// Web joystick concept - browser sliders/buttons produce bounded references.
// The ESP32-S3 sends compact commands to the STM32 controller.
// Example command family: #P15=<left_right>,P8=<forward_backward>!

void sendJoystickCommand(float forward, float turn) {
    forward = clamp(forward, -MAX_FORWARD_CMD, MAX_FORWARD_CMD);
    turn    = clamp(turn,    -MAX_TURN_CMD,    MAX_TURN_CMD);

    // P8 is interpreted as forward/backward speed reference.
    // P15 is interpreted as left/right yaw-rate reference.
    serial.printf("#P15=%.2f,P8=%.2f!", turn, forward);
}

// Design intent: the joystick changes references only.
// It does not bypass the inner balance loop that keeps the robot upright.
```

Source-code note

The excerpts above are the report-facing source listing for the core control path. The complete buildable source tree, including generated STM32CubeMX/HAL files, ESP32-S3 web interface files, and versioned development folders, is maintained in the project repository cited in Appendix D.

APPENDIX D - REFERENCES

- Green Thumb Box reference report, ME-GY 5643 Mechatronics Integrated Term Project - Group 2, Fall 2015: <https://mechatronics.engineering.nyu.edu/pdf/gtb-report.pdf>
- Apollo-kernel/mechatronics public repository and commit history: <https://github.com/Apollo-kernel/mechatronics>
- Project engineering log uploaded in this conversation: project report-Engineering Log of ROB-GY 5103 - Mechatronics (Section A).pdf
- Bosch Sensortec, BMI088 data sheet, document BST-BMI088-DS001.
- STMicroelectronics, STM32CubeMX product page and STM32Cube documentation for STM32 configuration and code generation.
- FreeRTOS official documentation for the real-time operating-system kernel used in the firmware architecture.
- M0603A motor reference PDFs included in the project repository.

Citation practice

Technical claims about the project build are based on the engineering log, repository commit history, and source-code inspection. Component background and part-selection rationale are supported by vendor documentation listed above.